to put belongs_to declarations is this: if a table has any columns whose values consist of ID values for another table (this concept is known by database designers as *foreign keys*), the corresponding model should have a belongs_to for each.

What do these various declarations do? Basically, they add navigation capabilities to the model objects. Because Rails added the belongs_to declaration to LineItem, we can now retrieve its Product and display the book's title:

```
li = LineItem.find(...)
puts "This line item is for #{li.product.title}"
```

To be able to traverse these relationships in both directions, we need to add some declarations to our model files that specify their inverse relations.

Open the cart.rb file in app/models, and add a call to has_many:

rails80/depot_f/app/models/cart.rb

```
class Cart < ApplicationRecord
»   has_many :line_items, dependent: :destroy
end
```

That has_many :line_items part of the directive is fairly self-explanatory: a cart (potentially) has many associated line items. These are linked to the cart because each line item contains a reference to its cart's ID. The dependent: :destroy part indicates that the existence of line items is dependent on the existence of the cart. If we destroy a cart, deleting it from the database, we want Rails also to destroy any line items that are associated with that cart.

Now that the Cart is declared to have many line items, we can reference them (as a collection) from a cart object:

```
cart = Cart.find(...)
puts "This cart has #{cart.line_items.count} line items"
```